

项目功能说明

1. 项目方向：游戏类
2. 项目梗概：本项目计划设计一个无限闯关的游戏，玩家扮演一位深入地牢探索的冒险家，与四位合作的探险者共同对抗地牢中的怪物。战斗结束后会根据玩家通关的层数作为玩家的成绩。
3. 具体功能：

1.Character 类：

一. 设计思想：

整个角色系统采用 面向对象的多态设计：

- 基类 Character 定义所有角色的通用属性和行为接口。
- 四个职业派生类（Warrior, Mage, Archer, Nonamer）分别实现不同职业的属性成长。
- 二十个具名英雄类（如 Leon, Anna, Shadow 等）继承自对应职业，并通过 addSkill 绑定专属技能，形成“职业模板 + 个性化技能”的组合模式。

二. 主要功能：

模块	关键方法/机制	说明
属性系统	attack, defense, health, mana, crittrate, critdamage, missrate, shield 等基础属性，以及配套的 xxxadd 变量	基础属性 + buff 加成实时叠加，getInfo()展示完整面板
Buff 系统	addbuff(), endTurn(), hasBuffType()	支持 20 多种状态（攻防增减、中毒、麻痹、无敌、吸血、穿透等），回合制自动计时与清除，addbuff 立即生效，endTurn 递减并回收
战斗系统	takeDamage(), calculateDamage(), calculateCounterDamage()	伤害计算流程：无敌免疫 → 闪避判定 → 护盾抵扣 → 防御减伤 → 最终伤害；支持暴击、穿透、吸血、防御反击
技能系统	Skill 基类 + healSkill/ buffSkill 派生类，useSkill()	技能消耗法力（可被 fixed 状态豁免），支持单体/群体治疗、增益/减益、控制效果
特殊机制	performSupportAttack()	弓箭手独有：当其他角色攻击时，弓箭手可发动 50% 伤害的协同攻击
装备系统	setWeapon(), removeWeapon()	装备提供百分比属性加成，并可附

		带特殊效果（如 fixed 免蓝、fastchange 速切）
存档系统	saveToJson(), loadFromJson()	使用 JSON 格式保存/读取角色等级、当前状态、buff 列表、装备名称，便于持久化

三、数据流与生命周期：

1. 创建：通过工厂函数 createCharacterByName()按名字实例化对应英雄，自动设置职业属性并添加专属技能。
2. 回合循环：startTurn()检查麻痹/睡眠 → 玩家选择技能 → useSkill()执行 → endTurn()处理 buff 衰减、中毒伤害等。
3. 销毁：析构函数自动释放所有动态分配的 Skill 对象，CharacterManager 负责统一管理角色对象的生命周期。

2.Weapon 类：

一、整体设计思路：

武器系统采用单层继承结构：

- 基类 Weapon：定义所有武器的通用属性(名称、稀有度、描述、各项加成系数)和序列化接口。
- 每个具体武器作为一个派生类（共 4 种稀有度 × 每种 5 个具体武器），在构造函数中设定专属的数值加成和特殊效果（specialEffects）。武器通过工厂函数 createWeaponByName()按字符串名称动态创建，便于从存档或配置中加载。

二、稀有度体系：

四种品质，数值强度递增：

稀有度	示例武器	特点
Common	木剑、魔法杖、木弓、木盾	单一属性小幅提升（×1.1~1.2），无特效
Rare	匕首、魔杖、硬木弓、骑士盾	双属性提升，部分带暴击伤害加成
Epic	斩龙者、毒蛇之牙、雷光魔杖...	显著提升（×1.3~1.5），多数带有特殊效果（中毒、麻痹、反击、护盾）
Legendary	诸神黄昏、命运之剑、弑神者...	极高提升（×1.5~2.0），特效强大（吸血、穿透、速切、无消耗、护盾）

三、特殊效果机制：

- 通过 buffSkill* specialEffects 指针挂载一个技能对象。

- 当角色装备该武器时，此 buffSkill 会在特定时机（通常是攻击后）自动触发，给目标或自身附加 buff。
- 常见特效：中毒、麻痹、吸血、护盾、穿透、速切、无消耗等。

四、存档与反序列化：

- toJson()仅保存武器名称（因为通过名称即可完全重建武器）。
- fromJson()目前为空实现，实际使用时需配合工厂函数 createWeaponByName()重新构造武器对象。
- 外部预定义了四个稀有度的武器名称列表（commonWeaponList 等），可用于随机掉落或商店刷新。

3.Enemy 类：

一、整体设计结构：

敌人系统与玩家角色共享相同的基类 Character，但在其基础上增加了敌人专属的类型枚举和 AI 决策逻辑

二、核心功能模块：

模块	关键内容	说明
难度分级	EnemyType 枚举：Normal/ Elite/ Boss	同一职业在不同难度下拥有差异化的基础属性（血量、攻击、防御等），Boss 数值最高
等级成长	每个敌人构造函数接受 level 参数（1~3）	等级越高，基础属性越强，同时暴击率/暴击伤害/闪避率也随等级提升
职业差异化	四个职业派生类分别设定属性模板	剑士高血高防、法师高攻低防、弓箭手中等输出、无名之人较均衡
技能配置	每个具体敌人在构造函数中创建三个技能：治疗、防御反击、专属 Buff	技能名称和效果因敌人而异（如狂暴剑士加攻击、血刃战士加吸血、大法师回蓝等）
AI 策略	AIStrategy()方法	基于血量判断： - 血量 < 50% 时，50%概率治疗，50%概率防御 - 否则 50%概率释放 Buff 技能 - 以上均不满足则普攻
继承关系	具体敌人 → 职业敌人 → Enemy → Character	复用角色系统的全部属性、Buff、伤害计算等逻辑，无需重复开发

4.Event 类：

一、整体设计结构：

事件系统由两个类组成：

EventWidget（继承自 QDialog，是一个模态对话框），EventManager（独立工具类，不继承 Qt 组件），两者分工明确：EventManager 负责“什么时候触发什么事件”，EventWidget 负责“把这个事件展示给玩家并处理玩家的选择”。

二、核心数据结构：

1. 事件类型枚举：Treasure// 神秘宝箱；Merchant// 旅行商人；Trap// 致命陷阱；Challenge// 强者挑战
四种事件各有不同的剧情描述和奖励池。
2. 奖励类型枚：StarUpgrade// 角色升星；HealAll// 全队回血；Gold// 金币；Weapon// 武器

三、核心功能模块：

模块	关键方法	说明
事件生成	generateEventContent()→ 调用四个 createXxxEvent()	根据事件类型随机生成奖励内容，同时更新 UI 文本
UI 展示	setEventInfo(), setEventImage()	设置标题、描述、图片，展示奖励预览
玩家交互	onAcceptClicked(), onRejectClicked()	接受→发射信号；拒绝→直接关闭
信号系统	rewardGranted, fightTriggered, eventRejected	三种信号供主界面连接，处理后续逻辑
缩放适配	resizeEvent()	手动等比缩放所有子控件，适配不同分辨率窗口

5.Prepare 类：

一、整体定位：

prepare 是游戏的核心运营界面，介于战斗之间。它负责：

- 管理角色/武器/卡牌的招募、摆放、合成、出售
- 配置出战阵容（最多 4 人）
- 提供羁绊预览、商店购物、情报查看
- 触发随机事件（宝箱/商人/陷阱/挑战）
- 生成敌人并根据层数决定难度
- 维护存档/读档

本质上是一个战前准备大厅，所有非战斗的养成与决策都在这里完成。

二、核心数据结构：

结构	说明
BenchItem	通用容器，可存放角色、武器或卡牌。包含类型、名称、图片路径、数据指针、星级、装备的武器指针等
QVector<BenchItem*> benchItems	备战席，固定 8 个槽位，空指针表示空

QVector<BenchItem*> fighters	对战席，固定 4 个槽位，仅能放置角色
FighterInfo	对战席角色信息的打包结构（角色对象 + 星级 + 武器），供战斗引擎使用

三、主要功能模块：

模块	关键方法	说明
备战席管理	addItemToBench, swapBenchItems, sellFromBench	增删改查，支持角色/武器/卡牌混合存放
对战席管理	moveToFighter, moveFromFighterToBench, swapFighters	上阵/下阵/换位，仅允许角色
武器系统	equipWeaponToBenchCharacter, equipWeaponToFighterCharacter	拖拽武器到角色身上完成装备，自动卸下旧武器
星级合成	swapBenchItems 中的合并逻辑	同名同星角色拖到一起自动升星（最高 5 星），同步提升角色属性
羁绊系统	updateBondEffects, refreshBondStats	统计出战席各职业数量，显示当前激活的羁绊效果（含组合羁绊）
敌人生成	generateEnemiesForFloor	根据层数、Boss/精英/普通规则，使用确定性种子生成敌人
事件系统	isEventFloor, on_btn_start_clicked 中的事件分支	20% 概率触发随机事件，通过 EventWidget 展示并处理奖励
卡牌使用	applyBuffCard, applyBuffCardToFighter	支持攻击法术、增益、事件、装备补给等多种卡牌类型
商店/情报	on_btn_shop_clicked, on_btn_review_clicked	跳转到商店购买或情报预览界面
存档系统	saveToFile, loadFromFile, saveGameState, loadGameState	使用 JSON 保存备战席/对战席所有物品及其装备关系、金钱、关卡数等
战斗流程	on_btn_start_clicked→ 生成敌人 → 创建 BattleEngine→ 打开 battle 窗口 → 处理胜负	胜利后推进关卡、恢复角色状态、自动存档；失败则重置进度

四、数据流与生命周期：

启动游戏 → prepare 初始化（加载存档或新建）

↓

玩家操作：购买/合成/上阵/装备

↓

点击「开始游戏」

└─ 20% 事件层 → EventWidget → 接受/战斗/拒绝

└─ 80% 战斗层 → 生成敌人 → battle 窗口

↓ 战斗结束

└─ 胜利：推进层数、恢复状态、存档、回到 prepare

└─ 失败：重置进度、回到主菜单

6. Battle 和 Battle_engine 类：

一、整体架构：

战斗系统由两个类协作完成：

Battle（战斗界面的 UI 呈现与交互），BattleEngine（战斗逻辑引擎，驱动状态机），两者通过 Qt 信号与槽通信：BattleEngine 发射 stateChanged、logAdded、uiNeedsUpdate、battleEnded 等信号，battle 接收后更新界面或做出响应。

二、核心功能模块：

1. 战斗状态机（BattleEngine）

采用有限状态机驱动回合流程，状态流转如下：DECISION_PHASE → PLAYER_ACTION → CHECK_DEATH1 → ENEMY_ACTION → CHECK_DEATH2 → ROUND_END → DECISION_PHASE

- DECISION_PHASE：等待玩家选择行动（普攻/技能/换人）。
- PLAYER_ACTION / ENEMY_ACTION：执行选定行动，包括普通攻击、技能释放、武器特效触发、协同攻击等。
- CHECK_DEATH1 / CHECK_DEATH2：检查前后排阵亡并自动换位，若一方全灭则进入 BATTLE_OVER。
- ROUND_END：调用 endTurn() 处理 buff 衰减、中毒伤害等，重置玩家选择状态。
- BATTLE_OVER：清除羁绊加成，发射 battleEnded 信号。

2. 伤害计算与战斗反馈

- 普通攻击伤害：calculateNormalAttackDamage() 基于攻击力、暴击率/暴击伤害计算，支持暴击。
- 技能执行：executeSkill() 根据 SkillTargetType 决定目标（自身、友方全体、敌方全体、敌方前台、指定友方单体等），调用 useSkill()。
- 武器特效：triggerWeaponAttackEffects() 在普攻后触发，支持中毒、麻痹、吸血、护盾、穿透、反击等。
- 协同攻击：triggerSupportArcherAttack() 在每次攻击后，检查后排存活的弓箭手，以 50% 伤害发动协同攻击。

- 防御反击：若目标处于 Defend 状态，takeDamage()会自动计算反击伤害。

3. 战斗 UI (battle)

- 布局：采用 QVBoxLayout+ QHBoxLayout+ QGridLayout 构建，包含玩家/敌人立绘、血条/法力条、状态标签、技能按钮（2×2 网格）、战斗日志。
- 动态更新：refreshUI()调用 updatePlayerUI()和 updateEnemyUI()，实时刷新血量、法力、护盾、buff 数量、羁绊加成等信息。
- 技能按钮状态：updateSkillButtons()根据法力值、fixed 状态禁用/启用技能按钮。
- 视觉反馈：playCombatFeedback()实现浮动伤害数字、立绘抖动、闪屏、颜色闪烁；triggerSkillEffects()根据技能名称关键词（火、冰、雷等）触发屏幕闪色。
- 目标选择：showTargetSelectionDialog()弹出自定义对话框，让玩家选择友方单体目标（用于指定治疗/增益技能）。
- 换人：change_clicked()调用 showTargetSelectionDialog(false)选择后排角色，交换到前排；若有 fastchangebuff 则不消耗回合。

4. 羁绊系统在战斗中的应用

applyBondEffects()在战斗开始时根据出战队伍的职业构成，自动为全队添加属性加成（生命、法力、攻击、防御、暴击、闪避等）。包含：

- 单职业羁绊（2 人/4 人）：法师加法力、剑士加生命、弓箭手加攻击、无名之人加暴击。
- 组合羁绊：四职同辉（全属性）、战法相济（攻击+法力）、疾影连射（暴伤+闪避）、铁壁神佑（生命+防御）。

战斗结束时通过 clearBondEffects()清除所有加成，避免影响下一场战斗。

三、数据流与生命周期：

玩家在 prepare 点击“开始战斗”

prepare 生成敌人、创建 BattleEngine（传入玩家队伍和敌人队伍）

battle 窗口打开，setEngine() 连接信号

BattleEngine 初始化：应用羁绊、随机先后手、发射 uiNeedsUpdate

玩家在 DECISION_PHASE 选择行动（普攻/技能/换人）

submitPlayerAction() / submitPlayerActionWithTarget()

状态机依次执行行动、检查死亡、回合结束，期间不断发射 logAdded / uiNeedsUpdate

战斗结束 → battleEnded 信号 → battle 关闭，发射 battleFinished 给 prepare

prepare 处理胜利/失败逻辑（推进关卡/重置进度）

7.Card 类：

一、整体设计结构：

卡片系统采用 三层继承 + 工厂模式，CardManager 作为总管家，负责所有卡牌的注册、生成、卡组管理、存档。

二、核心功能模块：

模块	关键类/方法	说明
卡牌基类	Card	持有名称、描述、目标角色、类型枚举；定义虚函数 use()
攻击卡	AttackCard	存储基础伤害、附加状态（如麻痹）、持续时间。use()调用目标 takeDamage() 并添加 buff
增益卡	BuffCard	存储 buff 类型、百分比数值、持续时间、是否全队。use()根据目标当前属性计算具体加成值（如攻击力+30%）并添加 buff
事件卡	EventCard	存储事件类型、奖励结构体 EventReward、是否需要战斗。本身不执行具体事件，而是将事件信息传递给 EventWidget 处理
装备卡	WeaponCard	支持两种构造方式：指定武器名称或随机品质。use()时若为随机则从对应品质列表中抽取一把武器，然后调用 target->setWeapon()
卡牌管理	CardManager	管理全局卡池 allCards 和玩家卡组 playerDeck；提供随机生成、按类型生成、起始卡组生成、存档读写
工厂函数	createCardByName	根据字符串名称创建对应的具体卡牌对象，解耦客户端与具体类

8.Review 类：

一、整体定位：

review 是一个战前情报预览界面，在玩家点击“开始战斗”之前展示下一层的信息。它有两种展示模式：

- 怪物层：显示即将面对的敌人阵容、属性、技能。
- 事件层：显示“？”符号，提示玩家前方是随机事件。

它位于 prepare 备战席与战斗之间，帮助玩家提前制定战术。

二、核心功能模块：

模块	关键方法/成员	说明
情报刷新	updateIntel(int currentFloor)	根据当前楼层数，判断下一层是事件层还是怪物层，并填充对应的 UI 控件
事件层展示	isEventFloor()调用	若为事件层，隐藏怪物图片，显示一个大的金色问号 + 描述文本“神秘房间”
怪物层展示	generateEnemiesForFloor()调用	调用 prepare 的敌人生成方法获取敌人列表，遍历显示头像、名称、HP/MP/攻击/防御、技能列表
返回按钮	on_btn_back_clicked()	发射 backToPrepare 信号，隐藏自身，返回备战席
自适应缩放	resizeEvent()	记录子控件初始几何，按设计基准（660×560）等比缩放
背景绘制	paintEvent()	绘制背景图（event_background.png），若无则填充深灰蓝色

9.Shop 类：

一、整体定位：

shop 是游戏中的商店界面，玩家可以在战斗间隙消耗金币购买角色、武器和卡牌。它与 prepare（备战席）紧密配合，购买的商品直接存入备战席。

二、核心功能模块：

模块	关键方法	说明
商品刷新	refreshCharacters()/ refreshWeapons()/ refreshCards()	从全局角色/武器/卡牌池中随机抽取 5 个，填充到对应 currentXxx 列表
商品展示	displayCharacters()/ displayWeapons()/ displayCards()	将 currentXxx 的数据渲染到 UI 的图片标签和文字标签上
交互处理	eventFilter()	拦截鼠标点击：点击图片→显示详情；点击名称→购买
商品详情	showDetail()	弹出 QDialog，展示商品图片、名称、价格、详细属性
购买逻辑	buyItem()	扣钱→标记售罄→addItemToBench()存入备战席
刷新费用	on_new_character_clicked()等	每次刷新花费 2 金币，调用对应的 refreshXxx()
退出商店	on_exitshop_clicked()	重置传说武器模式，发射 backToPrepare 信号，刷新备战席显示
货币显示	updateMoneyUI()/ notEnoughMoney()	实时更新金币余额；余额不足时显示 1 秒提示
传说武器模式	setLegendaryWeaponMode()	外部（如卡牌效果）可强制商店只刷传说武器（单价 100 金币）
自适应缩放	resizeEvent()	按设计基准（634×525）等比缩放所有子控件

10.其余类别:

包括声音, 总体风格类别, 以及一些技能特效类别, 还有主界面类别。

4. 相关分工:

黄负责 Prepare, Shop, review 类

陈负责 Battle, Battle_engine, 声音, 主题, 布局类

曾负责 Character, Enemy, Card, Event, Weapon 类的设计和图片, 主界面类以及各个类的存档功能

5. 总结与反思:

大项目加深了我们对与游戏制作从构想, 策划到落地的各个流程的认识, 强化了我们的程序设计能力和对 c 语言的理解, 锻炼了我们使用 qt 进行前后端开发的能力。

我们的游戏经过多个版本的修复和改进, 同时分发给多位玩家进行了持续将近半个月的测试, 经过了大量改进和优化, 包括代码和数值平衡以及美术层面, 也是我们心血的结晶。